

Query Optimiser for SJDB

COMP3211 Advanced Databases

Isha Girish Kulkarni (37642456)

igk1g25@soton.ac.uk

Overview

The optimiser transforms a canonical query plan, a project over a series of selects over cartesian products over scans, into an optimised left-deep join tree. The goal is to minimise the sizes of intermediate relations produced at each step, reducing overall query execution cost. Without optimisation, naive execution of cartesian products followed by selections can produce extremely large intermediate results that are expensive to process.

Cost Estimation

The Estimator implements the PlanVisitor interface and performs a depth-first traversal of the query plan. At each operator, it creates an output Relation with estimated tuple count $T(R)$ and attribute value counts $V(R,A)$ using the standard formulae:

- Scan: $T(\text{out}) = T(R)$, attributes and value counts copied directly from the named relation
- Product: $T(\text{out}) = T(R) \times T(S)$, all attributes from both sides retained unchanged
- Selection $\text{attr}=\text{val}$: $T(\text{out}) = T(R)/V(R,A)$, $V(A) = 1$
- Selection $\text{attr}=\text{attr}$: $T(\text{out}) = T(R)/\max(V(R,A),V(R,B))$, $V(A)=V(B)=\min(V(R,A),V(R,B))$
- Join: $T(\text{out}) = T(R) \times T(S)/\max(V(R,A),V(S,B))$, $V(A)=V(B)=\min(V(R,A),V(S,B))$, non-join attributes retain original value counts
- Projection: $T(\text{out}) = T(R)$, only projected attributes retained in the output relation

The constraint $V(R,A) \leq T(R)$ is automatically enforced by `Relation.addAttribute()`, ensuring estimates remain consistent.

Optimisation Strategy

The optimiser follows four steps:

Step 1 Decompose: The canonical plan tree is recursively traversed to extract all Scan operators, all Select predicates (both $\text{attr}=\text{val}$ and $\text{attr}=\text{attr}$), and the top-level Project attributes if present. This completely separates the query ingredients from the original unoptimised structure.

Step 2 Push selections down: For each Scan, any $\text{attr}=\text{val}$ predicate whose attribute belongs to that relation is immediately applied as a Select directly above the Scan. This reduces relation sizes before any joining occurs, making subsequent joins cheaper. Only $\text{attr}=\text{attr}$ predicates remain, becoming candidates for join conditions.

Step 3 Greedy join ordering: The remaining predicates become join conditions. The optimiser iterates over all pairs of current sub-plans, estimating each combination as a Join if a predicate connects them, or a Product otherwise. The pair with the smallest estimated output is selected and

merged. This repeats until one plan remains, preventing large intermediate relations and naturally converting Product and Select combinations into efficient Join operators.

Step 4 Re-apply projection: The top-level Project is wrapped around the final join tree, discarding all unnecessary columns from the output.

Limitations

The greedy approach is a heuristic and does not guarantee a globally optimal plan. It makes locally optimal choices at each step, which can occasionally miss a better overall join ordering. The cost model relies solely on tuple counts and attribute value counts, and does not account for physical factors such as indexes or buffer sizes. The estimator assumes uniform value distribution, which may not reflect real data skew, leading to estimation errors in practice.

Output

The following output was produced by running the provided Test harness. Both plans produce identical final results confirming correctness. The optimised plan eliminates the 15,000-row intermediate cartesian product entirely.

Canonical Plan (before)	Optimised Plan (after)
A in: A:100:a1,100:a2,15 out: 100:a1,100:a2,15	A in: A:100:a1,100:a2,15 out: 100:a1,100:a2,15
B in: B:150:b1,150:b2,100:b3,5 out: 150:b1,150:b2,100:b3,5	B in: B:150:b1,150:b2,100:b3,5 out: 150:b1,150:b2,100:b3,5
(A) TIMES (B) inl: 100:a1,100:a2,15 inr: 150:b1,150:b2,100:b3,5 out: 15000:a1,100:a2,15:b1,150:b2,100:b3,5	(A) JOIN [a2=b3] (B) inl: 100:a1,100:a2,15 inr: 150:b1,150:b2,100:b3,5 out: 1000:a1,100:a2,5:b1,150:b2,100:b3,5
SELECT [a2=b3] ((A) TIMES (B)) in: 15000:...	PROJECT [a2,b1] (...) in: 1000:a1,100:a2,5:b1,150:b2,100:b3,5 out: 1000:a2,5:b1,150
PROJECT [a2,b1] (...) in: 1000:a1,100:a2,5:b1,150:b2,100:b3,5 out: 1000:a2,5:b1,150	